CO2-AT1 - Analytical Problem

CSA0404 - OS

PRASANNA.A
192525130

1> CPU scheduling - Waiting Time & Turnaround Time

	BT
P1	10
P2	4
P3	6
P4	2

Gantt Chart

0 2 6 12 22

| P4 | P2 | P3 | P1 |

(a)

	WT	TAT
P4	0	2
P2	2	6
P3	6	12
P1	12	22

Avg. WT = $\frac{0+2+6+12}{4}$ = 5 ms

Avg TAT = $\frac{2+6+12+22}{4}$ = 10.5 ms

(c) • SJF reduces the average time from 11 to 5 ms

• It also reduces the average turn around time from 16.5 ms to 10.5 ms

• SJF mainly improves scheduling efficiency by reducing waiting and turn around time.

2> Round Robin Scheduling: [Time quantum = 4 ms]

	BT
P1	20
P2	5
P3	13
P4	8

Gantt chart :

| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 | P1 | P3 | P1 |

0 4 8 12 16 20 21 25 29 33 37 38 42 46

	CT	TAT	WT
P1	46	46	26
P2	21	21	16
P3	42	42	29
P4	29	29	21

Avg WT = $\frac{26+16+29+21}{4}$ = 23 ms

Avg TAT = $\frac{46+21+44+}{4}$ = 34.5 ms

(c). Fewer context switches occur, reducing scheduling overhead

- Response time for shorter process may increases.

---

3) Deadlock - Safety check using Banker's Algorithm

A=10 B=5 C=7

Need Matrix = Max - Allocation

	A	B	C
P1	7	4	3
P2	1	2	2
P3	6	0	0
P4	2	1	1
P5	5	3	1

A=7 B=2 C=5

A = 10-7 = 3

B = 5-2 = 3

C = 7-5 = 2

Available (3,3,2)

(b) P2 can execute → Available (5,3,2)

P4 → (7,4,3) P5 → (7,4,5) P1 → (7,5,5)

P3 → (10,5,7)

(c) P2 → P4 → P5 → P1 → P3

---

4) Deadlock Detection - Resource Allocation Graph:

$R_1 \rightarrow P_1$ (P_1 hold R_1), $P_3 \rightarrow P_1$ (P_3 request R_1)

$P_1 \rightarrow R_2$ (P_1 is requesting R_2)

$R_2 \rightarrow P_2$ is hold R_2)

$P_2 \rightarrow P_3$

$R_3 \rightarrow P_3$

$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$

∴ The System is in a deadlock

(c) Revovery Method

- Detect the deadlock
- Terminate one of the Processes
- Relearo the resource held by that Process
- The remaining Processes can then continue execution.

---

5) CPU utilization - Multiprogramming level Analysis

$U = 1 - (p^n)$ ∴ $p = 0.8$

where p = Probality of waiting for I/O

n = degree of multiprogramming

(a) $n = 4$ $U = 1 - (0.8)^4$

$U = 1 - 0.4096 = 0.5904$

CPU utilization = 59.04 %.

(b) $n = 6$

$U = 1 - (0.8)^6$

$U = 1 - 0.262144 = 0.737856$

(c) with 4 Processes, CPU utilization is 59.04 %.,

6 Processes, CPU utilization to 73.79 %.

Because it provides high CPU utilization and reduce Performance

6) Producer - consumer using semaphores

(a) Semaphore values

Operation	Empty	full	mutex
initial	6	0	1
P1	4	1	1
P2	3	2	1
P3	4	3	1
C1	5	2	1
C2	5	1	1

(b): • The mutex semaphore is Removed

• The empty semaphore prevented from adding items when the buffer is full

(c) This may lead to :

- Data corruption
- Incorrect buffer updates
- Race condition
- unpredictable Program behaviour

---

7) Process Synchronization - critical section:

- Process P_1 reads $x = 1000$
- Process P_2 also reads $x = 1000$
- P_1 increments and writes 1001
- P_2 increments and also writes 1001

(b) Total $1000 + 1000 = 2000$

Final value of x:

$1000 + 2000 = 2100$

Expected value of x

$= 2100$

Hardware-based solution

Test and Set (TSL) instruction (or compare and swap) to ensure only one process enter the critical at a time

6) Thread scheduling and performance:

Many to one	one to one
Many user threads map to one kernal thread	Each user thread maps to one kernal
only one thread executes at a time	Multiple threads can run in parallel
Lower overhead but no true	Better performance but hig

(b) • many to one: only one core is used to CPU utilization is low

• one to one: All 8 cores can executes threads

(c): one to one: All is the better model because it utilize all CPU cores, Provides true parallel exe

9) Monitor Synchronization:

(a) mutual Exclusion

A monitor allows only one process to access the shared printer at a time. Other processes must wait until printer becomes free.

(b) Role of wait() and signal()

→ suspends until the required condition becomes true.

→ wakes up a waiting process

(c) If signal() is omitted

waiting process will never be notified that the printer is free

10) Deadlock Prevention Vs Avoidance

(a) => Deadlock Prevention removes one of the four necessary conditions for deadlock

(b) Prevention: uses fixed rules to ensure deadlock never occurs

Avoidance: Check each resource request and grant it only

(c)

Prevention	Avoidance
Simpler to implement	More complex
Lower resource utilization	Better resource utilization
Lower system flexibility	Better overall performance